Contents

Rabin-Karp Pattern Matching — Complete Lecture Sheet **1**
1. Introduction and Goal of the Class . . . 1
2. The Problem . . . 2
3. The First Idea That Fails: ASCII Sum Hashing . . . 2
4. The Fix: Polynomial Hashing . . . 3
5. Step 1 — The Power Array . . . 4
6. Step 2 — The Prefix Hash Array of the Text . . . 5
7. Step 3 — The Pattern Hash . . . 6
8. Step 4 — Sliding the Window Over the Text . . . 6
9. Full Dry Run on `ababcabab` with Pattern `cab` . . . 7
10. Full C++ Program . . . 8
11. Expanded Notes on Steps That Were Passed Quickly in Class . . . 11
12. Hash Collisions and False Matches . . . 11
13. Complexity Analysis . . . 12
14. Course Guidance and Self-Practice (From This Class) . . . 12
15. One-Page Summary . . . 13

Rabin-Karp Pattern Matching — Complete Lecture Sheet

(C++ Version — Polynomial Hashing, Prefix Hash Array, and Sliding Window)

(This sheet follows the order in which the teacher worked through the topic on the board and in the transcript: the problem, the first idea that fails, the polynomial hash, the power array, the prefix hash array, the pattern hash, the sliding window, and finally complexity. Every step is shown in C++, the language used in class.)

---

1. Introduction and Goal of the Class

In this class the teacher introduced the **Rabin-Karp** algorithm, a method for **pattern matching**. Given a long **text** and a shorter **pattern**, the task is to find every position where the pattern occurs inside the text.

The naive way is to compare the pattern with the text one character at a time, at every possible starting position. The teacher's central idea is different:

> *"We won't compare character to character. We will compare hash values — integer with integer. To do that, we first have to generate the hash value."*

If the hash of every window of the text can be computed quickly, then each window is checked with **one integer comparison** instead of several character comparisons.

The class covered the topic in this order:

1. Define the problem with a concrete text and pattern.
2. Try the obvious hash (sum of ASCII values) and see why it fails.
3. Replace it with **polynomial hashing**, fixing two problems along the way (zero-valued letters and overflow).

4. Build the **power array**, the **prefix hash array** of the text, and the **pattern hash**.
5. Slide a window across the text and compare hashes.
6. Analyze the complexity, then discuss false matches.

---

2. The Problem

Text: `ababcabab` (9 characters, indices 0 to 8)

Pattern: `cab` (3 characters)

Question: At which starting indices does the pattern occur in the text?

Answer: Index **4**, because `text[4..6] = c, a, b`.

The Windows of Length 3

Since the pattern has length 3, the text can be divided into **windows** of length 3. The teacher wrote these windows on the board to show how many candidates there are:

Start index	Window	Start index	Window
0	`aba`	4	`cab`
1	`bab`	5	`aba`
2	`abc`	6	`bab`
3	`bca`		

There are `n - m + 1 = 9 - 3 + 1 = 7` windows. Each one must be checked against `cab`.

> **Naive approach:** For each of the 7 windows, compare up to 3 characters. In general, with a text of length `n` and a pattern of length `m`, this costs **O(n x m)** in the worst case. Rabin-Karp aims to reduce each window check to **O(1)**.

---

3. The First Idea That Fails: ASCII Sum Hashing

The first hash the class tried was the simplest one: **add the ASCII values of all characters** in the window.

Recall that `a = 97`, `b = 98`, `c = 99`.

Example 1: The teacher's board example `ab ab`:

```
97 + 98 + 97 + 98 = 390
```

Example 2: The string `baba`:

```
98 + 97 + 98 + 97 = 390
```

Both strings have the **same hash value** (390), but they are **different strings**. The hash ignores the **order** of the characters, so the sum cannot tell them apart.

A simpler example from the same discussion:

```
sum("aba") = 97 + 98 + 97 = 292
sum("aab") = 97 + 97 + 98 = 292
```

Strings with the same letters in a different order are called **anagrams**. Every anagram of a pattern produces the same sum, so the sum would report a match where none exists. The teacher's conclusion:

> *"Plain ASCII addition does not work here. The sum gives a false result. We need a hash that depends on the order of the characters."*

This is why the class moved from a **sum** to a **polynomial**.

---

4. The Fix: Polynomial Hashing

4.1 The Idea

Instead of adding the characters directly, give each position a **weight** that depends on its index. The weight is a power of a base `p`:

```
hash(s) = s[0] * p^0 + s[1] * p^1 + s[2] * p^2 + ... + s[m-1] * p^(m-1)
```

Because the weights differ by position, changing the order of the characters changes the result.

The teacher used `p = 10` on the board so the numbers are easy to read by hand:

```
hash("aba") = a * 10^0 + b * 10^1 + a * 10^2
hash("aab") = a * 10^0 + a * 10^1 + b * 10^2
```

Now the two anagrams get different values. The next two sections explain two details that must be right for this to work.

4.2 Why 'a' Must Be 1, Not 0

The natural first choice is to map each letter to `c - 'a'`, so that `a = 0`, `b = 1`, and so on. The teacher pointed out a problem with this:

```
"a"   -> 0
"aa"  -> 0 * 1 + 0 * 10 = 0
"aaa" -> 0 * 1 + 0 * 10 + 0 * 100 = 0
```

Three different strings now have hash 0. A letter with value 0 contributes nothing, so strings made only of `a` cannot be told apart by length.

Fix: map each letter to `c - 'a' + 1`, so that `a = 1`, `b = 2`, ..., `z = 26`.

```
long long val(char c)
{
    return c - 'a' + 1;
}
```

With this mapping, `"a"` gives 1, `"aa"` gives 11, and `"aaa"` gives 111. They are all different.

4.3 Choosing the Base and the Modulus

The base `p`:

- The teacher used `p = 10` for hand calculation, because the numbers stay readable.
- However, base 10 is **too small** for the letter values, which go up to 26. Two different strings can then collide. For example, with `k = 11`, the string `"ka"` gives `11 + 1 * 10 = 21`, and the string `"ab"` gives `1 + 2 * 10 = 21`. Two different strings, one hash.
- In practice, a **prime** base larger than the letter values is used. The class recommended **31 or 37**. The code below uses **31**.

The modulus `M`:

- The powers grow very fast. For a text of length 100, `p^100` is far too large to store in any integer type.
- Therefore every value is kept **modulo a large prime**. The class used `M = 1,000,000,007` (that is, `10^9 + 7`).

4.4 Overflow Control

Because all values are reduced modulo `M`, every intermediate value stays small. When two values below `M` are multiplied, the product is at most about `10^18`, which fits in a `long long` (maximum about `9.2 x 10^18`). So the rule is:

> **Use `long long`, and take `% MOD` after every multiplication and every addition.**

---

5. Step 1 — The Power Array

Why It Is Needed

Every window calculation needs the powers `p^0, p^1, p^2, ...` up to the length of the text. The teacher noticed that calling a power function again and again inside the loops adds **function call overhead**. Instead, the powers are computed **once** and stored in an array, and then accessed by index.

> *"Instead of calling the power function every time, generate the powers once, and access them as an array."*

How It Is Built

- `pw[0]` is always **1**, because any base to the power 0 equals 1.
- Each next entry is the previous entry multiplied by the base.

```
vector<long long> pw(n);
pw[0] = 1;
for (int i = 1; i < n; i++)
{
    pw[i] = pw[i - 1] * P % MOD;
}
```

Power table for the base 10 (used in the hand calculation):

Index `i`	0	1	2	3	4	5	6	7	8
`pw[i]`	1	10	100	1000	10000	100000	1000000	10000000	100000000

---

6. Step 2 — The Prefix Hash Array of the Text

The Idea

Build an array `hs`, where `hs[i]` is the hash of the text from index 0 up to index `i`:

```
hs[0] = val(text[0])
hs[i] = hs[i - 1] + val(text[i]) * pw[i]
```

The teacher connected this directly to the **prefix sum** technique from an earlier class. A prefix sum keeps a running total. Here the running total is a **weighted** running total, where each character is multiplied by its position's power.

The Code

```
vector<long long> hs(n);
hs[0] = val(text[0]);
for (int i = 1; i < n; i++)
{
    hs[i] = (hs[i - 1] + val(text[i]) * pw[i]) % MOD;
}
```

Hand Calculation for `ababcabab` (Base 10)

The table below shows the values without the modulus, since they stay small for this text:

`i`	`text[i]`	`val`	`pw[i]`	`val * pw[i]`	`hs[i]`
0	a	1	1	1	1
1	b	2	10	20	21
2	a	1	100	100	121
3	b	2	1000	2000	2121
4	c	3	10000	30000	32121
5	a	1	100000	100000	132121
6	b	2	1000000	2000000	2132121
7	a	1	10000000	10000000	12132121
8	b	2	100000000	200000000	212132121

Reading one row: at `i = 4`, the character `c` has value 3, its weight is `10^4`, so it adds `30000`. The previous total was `2121`, and the new total is `2121 + 30000 = 32121`.

---

7. Step 3 — The Pattern Hash

The pattern is hashed the same way, using the same power array, but only over the pattern's own length `m`:

```
long long patHash = 0;
for (int i = 0; i < m; i++)
{
    patHash = (patHash + val(pattern[i]) * pw[i]) % MOD;
}
```

For the pattern `cab` (base 10):

```
c * 10^0 + a * 10^1 + b * 10^2
= 3 * 1 + 1 * 10 + 2 * 100
= 3 + 10 + 200
= 213
```

So `patHash = 213`. (With the base 31 used in the program, the same pattern gives `3 + 1 * 31 + 2 * 961 = 1956`. The method is the same; only the base changes.)

---

8. Step 4 — Sliding the Window Over the Text

The Key Formula

Take a window that starts at index `l` and has length `m`. It ends at index `r = l + m - 1`. The window hash is obtained from the prefix array by subtraction:

```
window_raw = hs[r] - hs[l - 1]        (if l > 0)
window_raw = hs[r]                    (if l = 0)
```

Why the Subtraction Works

`hs[r]` contains the contributions of **all characters from index 0 to r**. `hs[l - 1]` contains the contributions of indices 0 to `l - 1`. Subtracting removes the unwanted left part, and what remains is the contribution of indices `l` to `r`:

```
window_raw = text[l] * p^l + text[l+1] * p^(l+1) + ... + text[r] * p^r
           = p^l * (text[l] * p^0 + text[l+1] * p^1 + ... + text[r] * p^(m-1))
           = p^l * (hash of the window)
```

So the raw window value is the window's own hash **multiplied by `p^l`**. This leads to the comparison rule:

> **A window matches the pattern if and only if `window_raw == patHash * p^l`.**

Why We Multiply the Pattern Instead of Dividing the Window

A reader might expect the code to divide `window_raw` by `p^l` to get the window's hash. The teacher advised against this:

> *"Division causes precision loss. Instead, multiply the pattern hash by `p^i` and compare. Everything stays an integer."*

Multiplication keeps every value exact. Division would produce fractions, and under modular arithmetic division would also need extra machinery (a modular inverse).

Why the First Window Is a Special Case

When `l = 0`, the formula `hs[l - 1]` would be `hs[-1]`, which does not exist. The teacher therefore handled `i = 0` separately: there is nothing to subtract, so the window hash is `hs[r]` directly.

```
long long winHash = hs[r];
if (i > 0)
{
    winHash = (winHash - hs[i - 1] + MOD) % MOD;
}
```

The `+ MOD` keeps the result non-negative before the `% MOD`, because a subtraction in modular arithmetic can produce a negative value in C++.

The Loop Bound

The window must fit inside the text. The starting index `i` runs from `0` to `n - m`:

```
for (int i = 0; i + m <= n; i++)
```

This is the same as `i < n - m + 1`, which gives exactly `n - m + 1` windows.

---

9. Full Dry Run on `ababcabab` with Pattern `cab`

The table below uses the **base 10** numbers from the hand calculation. For each window start `i`, it shows the value the code compares (`hs[r] - hs[l-1]`) and the value it must match (`patHash * p^i`).

i	Window	hs[r] - hs[l-1]	213 * p^i	Match?
0	aba	121	213	No
1	bab	2120	2130	No
2	abc	32100	21300	No
3	bca	132000	213000	No
4	cab	2130000	2130000	**Yes**
5	aba	12100000	21300000	No
6	bab	212000000	213000000	No

Working out the match at `i = 4`:

```
window_raw = hs[6] - hs[3]
           = 2132121 - 2121
           = 2130000

patHash * p^4 = 213 * 10000
              = 2130000
```

```
2130000 == 2130000   -->  match at index 4
```

Checking one non-match, `i = 2`:

```
window_raw = hs[4] - hs[1] = 32121 - 21 = 32100
patHash * p^2 = 213 * 100 = 21300
32100 != 21300   -->  no match
```

The class version of the program stops at the **first** match. The teacher noted this as a design choice, and suggested that the indices could also be stored in an array if all matches are needed (see Section 10.2).

---

10. Full C++ Program

10.1 Class Version (Reports the First Match)

```cpp
#include <bits/stdc++.h>
using namespace std;

const long long P   = 31;            // base (prime, larger than 26)
const long long MOD = 1000000007LL;  // large prime modulus

// letter -> number: a = 1, b = 2, ..., z = 26  (never 0)
long long val(char c)
{
    return c - 'a' + 1;
}

int main()
{
    string text, pattern;
    cin >> text >> pattern;

    int n = text.size();
    int m = pattern.size();

    if (m > n)
    {
        cout << "Pattern not found";
        return 0;
    }

    // Step 1: power array, pw[i] = P^i mod MOD
    vector<long long> pw(n);
    pw[0] = 1;
    for (int i = 1; i < n; i++)
    {
```

```
        pw[i] = pw[i - 1] * P % MOD;
    }

    // Step 2: prefix hash array of the text
    vector<long long> hs(n);
    hs[0] = val(text[0]);
    for (int i = 1; i < n; i++)
    {
        hs[i] = (hs[i - 1] + val(text[i]) * pw[i]) % MOD;
    }

    // Step 3: hash of the pattern
    long long patHash = 0;
    for (int i = 0; i < m; i++)
    {
        patHash = (patHash + val(pattern[i]) * pw[i]) % MOD;
    }

    // Step 4: slide the window over the text
    for (int i = 0; i + m <= n; i++)
    {
        int r = i + m - 1;

        long long winHash = hs[r];
        if (i > 0)
        {
            winHash = (winHash - hs[i - 1] + MOD) % MOD;    // remove the left part
        }

        // compare with patHash * P^i  (multiply, never divide)
        if (winHash == patHash * pw[i] % MOD)
        {
            cout << "Pattern found at index " << i;
            return 0;
        }
    }

    cout << "Pattern not found";
    return 0;
}
```

Sample input:

```
ababcabab
cab
```

Expected output:

```
Pattern found at index 4
```

10.2 Variant: Report Every Match

The teacher mentioned that the indices of all matches can be stored in an array instead of stopping at the first one. Only the last part of `main()` changes:

```
vector<int> matches;

for (int i = 0; i + m <= n; i++)
{
    int r = i + m - 1;

    long long winHash = hs[r];
    if (i > 0)
    {
        winHash = (winHash - hs[i - 1] + MOD) % MOD;
    }

    if (winHash == patHash * pw[i] % MOD)
    {
        matches.push_back(i);
    }
}

if (matches.empty())
{
    cout << "Pattern not found";
}
else
{
    cout << "Pattern found at indices:";
    for (int idx : matches)
    {
        cout << " " << idx;
    }
}
```

For `ababcabab` and `cab`, the output is `Pattern found at indices: 4`.

10.3 Using the Same Program for Other Patterns

Practice with the same text and different patterns (the expected answers are given so you can check your hand calculation):

Pattern	Matches at indices
`aba`	0 and 5
`bab`	1 and 6
`abc`	2
`cab`	4
`zzz`	none

---

11. Expanded Notes on Steps That Were Passed Quickly in Class

The following steps were mentioned briefly during the class. They are expanded here, because each one is needed for the algorithm to be correct.

11.1 Why the Sum Hash Is Order-Blind

The sum of the characters is a **symmetric** function: changing the order of the characters does not change the result. Any hash that treats the characters symmetrically (sum, product, XOR of letters) will collide on anagrams. The polynomial hash avoids this by giving each position a different weight.

11.2 Why the Prefix Array Is Built Once

The text is scanned once to build `hs`. After that, every window hash costs one subtraction. Without the prefix array, each window hash would have to be recomputed from scratch in `m` steps, which would bring back the naive cost.

11.3 Why `hs[i]` Multiplies by `pw[i]` and Not by `pw[i - 1]`

The character at position `i` must carry the weight `p^i`, the same weight it has in the pattern hash when it sits at position `i`. If the weights were shifted, the window values would not line up with the pattern value.

11.4 The Role of the Modulus in Subtraction

Modular subtraction can produce a negative number in C++ (for example `(3 - 5) % 7` gives `-2`). Adding `MOD` before taking the remainder gives the correct non-negative value. Forgetting this step is a common bug.

11.5 Why Only Window Starts Up to `n - m` Are Checked

A window starting at `i > n - m` would extend beyond the end of the text, so it cannot contain the whole pattern. The loop bound prevents this.

11.6 Why the Power Lookup Is Also Used for the Pattern

The pattern hash uses `pw[0]` through `pw[m - 1]`, and the comparison uses `pw[i]`. Since `m <= n`, the power array (of length `n`) always has enough entries.

---

12. Hash Collisions and False Matches

A hash match is a **strong signal**, but in general it is not a **proof**. Two different strings can produce the same hash value. This is called a **collision**, and it leads to a **false match**.

- With a large prime modulus and a good base, collisions are rare for typical data. The class described the chance of a false match as very small, but not zero.

- Some implementations **verify** a reported match by comparing the `m` characters directly. This costs `O(m)` only when a match is reported, so it does not change the average running time. The class code did not include this check.
- For the example in this sheet, no false match can occur. The reason: the difference between any window and the pattern is a polynomial whose coefficients lie between -25 and 25. Its absolute value is at most `25 * (1 + 31 + 961) = 24825`, which is smaller than the modulus `1,000,000,007`. Since the modulus is prime and does not divide 31, that difference can be zero modulo `M` only if it is exactly zero. So the only window that matches is one where the strings are truly equal.

---

13. Complexity Analysis

Time

Step	Work	Time
Power array	`n` multiplications	O(n)
Prefix hash of text	`n` multiplications and additions	O(n)
Pattern hash	`m` multiplications and additions	O(m)
Sliding window	`n - m + 1` windows, each one subtraction and one multiplication	O(n - m + 1) = O(n)
Total		**O(n + m)**

Comparison with the Naive Approach

Approach	Time per window	Total time
Naive character comparison	up to `m` character comparisons	O(n x m)
Rabin-Karp (polynomial hash)	1 integer comparison	O(n + m)

Space

The two arrays `pw` and `hs` each have length `n`, so the space is **O(n)**.

Worst Case Remark

If many windows collide with the pattern hash and each is verified character by character, the work can grow toward `O(n x m)` again. With a good base and a large prime modulus this is very unlikely, which is why the average behavior is O(n + m).

---

14. Course Guidance and Self-Practice (From This Class)

- **Build the hash by hand before writing code.** The teacher's approach was to write `hs` for a short text on paper, then check the window values. Do this for `ababcabab` before running the program.

- **Understand the three reasons behind the three details:** `val` starts from 1 (to avoid zero-valued letters), the base must be larger than the letter values (to avoid collisions), and the modulus keeps values small (to avoid overflow).
- **Explain why the code multiplies the pattern hash and does not divide the window.** This is a common theory question, and the answer is precision (and, under modular arithmetic, the need for a modular inverse).
- **Practice the full algorithm by hand** on at least two patterns from Section 10.3 (for example `aba` and `bab`), and confirm the expected indices.
- **Derive the window formula yourself:** start from `hs[r]` and explain, in your own words, why `hs[r] - hs[l-1]` equals `p^l` times the window's hash.
- **Compare with the naive method.** Count how many character comparisons the naive method makes for `ababcabab` and `cab`, and compare with the single integer comparison per window in Rabin-Karp.

---

15. One-Page Summary

Rabin-Karp — Pattern Matching With Polynomial Hashing

Goal: find all positions where a pattern of length `m` occurs in a text of length `n`, using one integer comparison per window.

Hash function (for a string `s` of length `m`):

```
hash(s) = val(s[0]) * P^0 + val(s[1]) * P^1 + ... + val(s[m-1]) * P^(m-1)   (mod M)
val(c)  = c - 'a' + 1          (a = 1, never 0)
P = 31 (prime, larger than the letter values),  M = 1,000,000,007
```

The four steps:

```
1. pw[i]  = P^i mod M                                (power array, built once)
2. hs[i]  = hs[i-1] + val(text[i]) * pw[i]           (prefix hash of the text)
3. patHash = sum of val(pattern[i]) * pw[i]          (hash of the pattern)
4. for each window start i = 0 .. n-m:
       winHash = hs[i+m-1] - hs[i-1]   (if i > 0; else hs[m-1])
       if winHash == patHash * pw[i]  -> match at index i
```

Key points to remember:

- A window's raw value equals `P^i` times its own hash, so the pattern is multiplied by `P^i` (never the window divided).
- The first window (`i = 0`) has nothing to subtract, so it is handled as a special case.
- Subtraction must add `MOD` before the `% MOD`, to stay non-negative.
- Letters start at 1, not 0, so that strings like `"a"`, `"aa"`, `"aaa"` have different hashes.
- A hash match is a strong signal; in general, verifying the characters removes any doubt.

Complexity: O(n + m) time, O(n) space. The naive method is O(n x m).

Quick Recap Table

Concept	Purpose
Sum hash (ASCII)	Fails: ignores order, so anagrams like `aba` and `aab` collide
Polynomial hash	Weights each character by `P^i`, so order matters
`val(c) = c - 'a' + 1`	Ensures no letter has value 0
Prime base P (31)	Keeps distinct strings from colliding; base 10 is for hand calculation only
Modulus M	Keeps every value small and prevents overflow
Power array `pw`	Precomputed powers, avoids repeated function calls
Prefix hash array `hs`	Running weighted total, lets any window hash be found with one subtraction
`hs[r] - hs[l-1]`	Extracts the window's contribution, equal to `P^l` times the window hash
`patHash * pw[i]`	Compares without division, so no precision is lost
Special case `i = 0`	Avoids reading `hs[-1]`, which does not exist